

# Introduction into NAET

---



David A. Birkenmayer

AG Optimierung, RPTU Kaiserslautern

# What is L $\exists\forall$ N?

---



Lean is...

- a *functional programming language*

# What is L $\exists\forall$ N?

---



Lean is...

- a *functional programming language*
- *statically* and *dependently typed*

# What is L $\exists$ N?

---



Lean is...

- a *functional programming language*
- *statically* and *dependently typed*
- a *theorem prover*

## Python — imperative

```
def total(xs):  
    s = 0  
    for x in xs:  
        s = s + x  
    return s
```

## Lean — functional

```
def total (xs : List Int) : Int :=  
  match xs with  
  | []      => 0  
  | x :: ys => x + total ys
```

|              | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|--------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|              | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is | a state that is changed                                                                             | an expression that is built                                                                                                        |

|                                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|--------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is<br>programming is | a state that is changed<br>giving a sequence of instructions                                        | an expression that is built<br>applying and composing functions                                                                    |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |
| repetition is  | a loop: <code>for</code> , <code>while</code>                                                       | recursion                                                                                                                          |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |
| repetition is  | a loop: <code>for</code> , <code>while</code>                                                       | recursion                                                                                                                          |
| a function is  | a subroutine                                                                                        | a value with a type                                                                                                                |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |
| repetition is  | a loop: <code>for</code> , <code>while</code>                                                       | recursion                                                                                                                          |
| a function is  | a subroutine                                                                                        | a value with a type                                                                                                                |
| a function may | have side-effects                                                                                   | only map input to output ( <i>pure</i> )                                                                                           |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |
| repetition is  | a loop: <code>for</code> , <code>while</code>                                                       | recursion                                                                                                                          |
| a function is  | a subroutine                                                                                        | a value with a type                                                                                                                |
| a function may | have side-effects                                                                                   | only map input to output ( <i>pure</i> )                                                                                           |
| you gain       | speed and control                                                                                   | the ability to <i>prove</i> what it computes                                                                                       |

|                | Python — imperative                                                                                 | Lean — functional                                                                                                                  |
|----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| a program is   | a state that is changed                                                                             | an expression that is built                                                                                                        |
| programming is | giving a sequence of instructions                                                                   | applying and composing functions                                                                                                   |
| variables are  | mutable <code>x = x + 1</code>                                                                      | immutable                                                                                                                          |
| repetition is  | a loop: <code>for</code> , <code>while</code>                                                       | recursion                                                                                                                          |
| a function is  | a subroutine                                                                                        | a value with a type                                                                                                                |
| a function may | have side-effects                                                                                   | only map input to output ( <i>pure</i> )                                                                                           |
| you gain       | speed and control                                                                                   | the ability to <i>prove</i> what it computes                                                                                       |

Nothing is assigned, nothing is repeated, there is no state to carry. The Lean version *is* the recursive definition you would write on a blackboard — the Python version is correct too, but to see **that** you have to run it in your head.

## Python — dynamically typed

```
def total(xs):  
    s = 0  
    for x in xs:  
        s = s + x  
    return s
```

## Lean — statically typed

```
def total (xs : List Int) : Int :=  
  match xs with  
  | []      => 0  
  | x :: ys => x + total ys
```

|                   | Python — dynamically typed                                                                          | Lean — statically typed                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                   | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| types are checked | at run-time                                                                                         | at compile-time                                                                                                                    |

## Python — dynamically typed

```
def total(xs):  
    s = 0  
    for x in xs:  
        s = s + x  
    return s
```

## Lean — statically typed

```
def total (xs : List Int) : Int :=  
    match xs with  
    | []      => 0  
    | x :: ys => x + total ys
```

types are checked

at run-time

at compile-time

a type error is

a crash, on the unlucky input

a compilation error

|                   | Python — dynamically typed                                                                          | Lean — statically typed                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                   | <pre>def total(xs):<br/>    s = 0<br/>    for x in xs:<br/>        s = s + x<br/>    return s</pre> | <pre>def total (xs : List Int) : Int :=<br/>    match xs with<br/>      []      =&gt; 0<br/>      x :: ys =&gt; x + total ys</pre> |
| types are checked | at run-time                                                                                         | at compile-time                                                                                                                    |
| a type error is   | a crash, on the unlucky input                                                                       | a compilation error                                                                                                                |
| a type is         | an optional hint, ignored by the interpreter                                                        | a promise the compiler <i>enforces</i>                                                                                             |

# LEAN is statically typed



This is a second, independent axis: C is statically typed and imperative, Python is dynamically typed and imperative.

This is a second, independent axis: C is statically typed and imperative, Python is dynamically typed and imperative.

Ordinary static type systems stop at “is a list”. Once a type can say “is a sorted list of length  $n$ ”, it can also say *“is a proof that  $n$  is even”* — and that is the rest of this talk.

# LEAN is dependently typed

---



- every object has a type
- type-correctness is checked at compile-time
- *types are themselves values* (types are first-class citizens)
- types can *depend* on values
- types themselves can do computation

# LEAN is dependently typed

---



- every object has a type
- type-correctness is checked at compile-time
- *types are themselves values* (types are first-class citizens)
- types can *depend* on values
- types themselves can do computation

**blah:**

# LEAN is dependently typed



- every object has a type
- type-correctness is checked at compile-time
- *types are themselves values* (types are first-class citizens)
- types can *depend* on values
- types themselves can do computation

**blah:**

**blah:**

# Lean is a theorem prover

---



-

# Lean is a theorem prover

---



- 

**blah:**

# Lean is a theorem prover

---



- 

**blah:**

**blah:**

---

# Bibliography

---

# Thank you for your attention!

Shoutout to Fabian vdW for the Slide design